biomimetics

MDPI

Article

Motion Strategy Generation Based on Multimodal Motion Primitives and Reinforcement Learning Imitation for Quadruped Robots

Qin Zhang [1,*], Guanglei Li [1], Benhang Liu [1], Chenxi Li [1], Chuanle Zhu [2] and Hui Chai [2]

[1] School of Electrical Engineering, University of Jinan, Jinan 250022, China
[2] School of Control Science and Engineering, Shandong University, Jinan 250061, China
* Correspondence: cse_zhangq@ujn.edu.cn

Abstract

With the advancement of task-oriented reinforcement learning (RL), the capability of quadruped robots for motion generation and complex task completion has significantly improved. However, current control strategies require extensive domain expertise and time-consuming design processes to acquire operational skills and achieve multi-task motion control, often failing to effectively manage complex behaviors composed of multiple coordinated actions. To address these limitations, this paper proposes a motion policy generation method for quadruped robots based on multimodal motion primitives and imitation learning. A multimodal motion library was constructed using 3D engine motion design, motion capture data retargeting, and trajectory planning. A temporal domain-based behavior planner was designed to combine these primitives and generate complex behaviors. We developed a RL-based imitation learning training framework to achieve precise trajectory tracking and rapid policy deployment, ensuring the effective application of actions/behaviors on the quadruped platform. Simulation and physical experiments conducted on the Lite3 quadruped robot validated the efficacy of the proposed approach, offering a new paradigm for the deployment and development of motion strategies for quadruped robots.

Keywords: imitation learning; behavior generation; action design; quadruped robot

Check for updates

Academic Editor: Xiaojie Wang

Received: 4 December 2025
Revised: 20 January 2026
Accepted: 29 January 2026
Published: 4 February 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Quadruped robots exhibit exceptional terrain adaptability, demonstrating broad application prospects in disaster rescue, environmental inspection, and related fields [1–3]. A representative example is Boston Dynamics' Spot, which has shown outstanding performance across diverse environments, including construction sites and disaster zones [4]. However, existing research predominantly focuses on unimodal policy transfer, failing to effectively integrate the complementary advantages of different motion generation methods [5]. Consequently, establishing a cross-modal collaborative framework to achieve motion strategy fusion and adaptive selection is pivotal for advancing quadruped locomotion intelligence.

To enable effective motion strategy fusion, a systematic evaluation of existing motion generation approaches and their limitations is essential. Current research focuses on two primary methodologies: trajectory planning-based and motion capture-based methods.

Trajectory planning-based motion generation aims to produce optimal motion trajectories that satisfy smoothness, feasibility, and energy efficiency under multiple constraints.

Medeiros et al. [6] employed nonlinear programming to co-optimize base/wheel positions, interaction forces, and terrain information for wheeled-legged robots. Liu et al. [7] proposed a hierarchical framework combining front-end safety search, B-spline convex hull optimization, and iterative refinement, achieving 100% navigation success in static cluttered environments while reducing energy consumption. Song et al. [8] focused on energy-optimal jumping trajectory planning, enabling robots to overcome complex obstacles.

Motion data-driven generation methods leverage biological motion characteristics to create robust, lifelike, and generalizable quadruped motions. Ju et al. [9] pioneered the cross-validation of spiral theory stability models with biological gait data, systematically revealing the dynamic advantages of common gait sequences. Yao et al. [10] developed a video-based biomimetic adaptation network, using deep learning to extract spatiotemporal key features from animal motions and transferring them via a motion adapter. Motion video tracking captures actions from videos and extracts corresponding trajectories for motion generation. Additionally, motion capture technology provides high-precision motion data. Li et al. [3] adopted multimodal motion primitive encoding to decouple cross-scale motion features from canine multi-terrain motion capture data. Fawcett et al. [11] developed a data-driven template-based hierarchical control method for the real-time planning and control of dynamic quadruped robots.

In recent years, reinforcement learning (RL)-based motion control has emerged as a unified framework for robotic locomotion [12]. RL has become a promising paradigm for developing robust legged movement control strategies [13–15], enabling agents to learn motion generation policies directly through environmental interactions [16]. Remarkable achievements include agile behaviors such as balancing, running, jumping, and robust walking under environmental uncertainties [17]. Hwangbo et al. [18] established a sim-to-real transfer framework with data-driven actuator modeling. Bellegarda et al. [19] addressed unstructured terrain disturbances via a hybrid RL framework for dynamic jumping control. Azimi and Hoseinnezhad [20] proposed a hierarchical RL framework to enhance the stability and adaptability of quadruped robots in dynamic environments.

Recent studies explore integrating imitation learning into RL to reduce reward design and unnatural behaviors. Peng et al. [21] pioneered a primitive-fused deep RL paradigm, constructing a bio-inspired transfer framework for cross-domain animal-to-robot motion style conversion. To address skill generalization challenges, Yang et al. [22] proposed a biomimetic motion primitive learning framework with heterogeneous reward mechanisms, enabling robots to acquire diverse skills through imitation learning. Roh [23] designed a ground reaction force (GRF)-based reward function for animal motion imitation, achieving dynamic speed transitions during galloping and validating the efficacy of bio-inspired strategies for dynamic performance optimization. Chen et al. [24] introduced an end-to-end torque control RL paradigm, directly outputting joint torques instead of traditional position control, demonstrating superior anti-disturbance capabilities and reward maximization. Wang et al. [25] abandoned static control for load-carrying quadruped manipulators, proposing an RL-based arm–body dynamic coordination method inspired by quadruped limb synergies, significantly improving disturbance rejection. Miki et al. [26] fused vision and proprioception via gated attention mechanisms, reducing terrain misclassification during Alpine field tests while achieving 0.8 m/s locomotion speeds—their dynamic weighting mechanism offers a novel paradigm for cross-modal collaborative control. Similarly, Ding et al. [27] proposed a vision–language–action model, enabling quadruped robots to perform complex tasks in diverse environments with enhanced adaptability. From the perspective of system modeling assumptions and prior information, existing motion generation and control methods exhibit different trade-offs among interpretability, flexibility, and engineering practicality. Trajectory planning approaches rely on explicit dynamic models and

constraints, offering strong interpretability but limited flexibility in complex multi-task scenarios [28]. Reinforcement learning methods optimize policies through reward-driven learning and demonstrate strong adaptability; however, they typically require carefully designed reward functions and extensive interaction data, and their training stability and generalization performance remain challenging in real-world applications [29]. In contrast, imitation learning introduces expert demonstrations as prior knowledge, providing an effective inductive bias for policy search and constraining the optimization process within a reasonable motion manifold [30]. As a result, imitation learning improves training efficiency while maintaining stability and engineering feasibility for complex behavior generation.

Current research on optimizing the locomotion capabilities of quadruped robots often faces challenges, such as limited dimensionality in motion generation, abrupt transitions during behavior composition, and constrained control optimization objectives. These limitations hinder the reliable and efficient execution of smooth movements and multi-task operations in complex scenarios. To address these issues, this paper proposes a motion strategy generation method for quadruped robots based on multimodal motion primitives and imitation learning. The multimodal motion primitives do not refer to multiple motion primitives learned within a unified parameter space. Instead, they denote a collection of heterogeneous action representations derived from distinct motion generation paradigms, including 3D-engine-based keyframe specification, motion primitives obtained via motion capture data retargeting, and analytically generated trajectories based on central pattern generators (CPGs).

Existing approaches are typically trained for a single motion pattern, which makes it difficult to achieve the integrated execution of heterogeneous behaviors—such as stepping in place, locomotion, and posture adjustment—within a unified control framework [18]. In contrast, the proposed method enables a unified representation and seamless switching among multiple behaviors through multimodal motion modeling and a behavior planning mechanism. The main contributions of this paper are as follows:

- A fundamental motion primitive library for quadruped robots was designed, establishing an underlying behavioral foundation for executing complex tasks and enabling flexible motion control.
- A modular architecture was employed to achieve spatiotemporal encoding of motion primitives and skill-chain recombination for quadruped robots, enabling the dynamic synthesis of behavior sequences in complex scenarios through a behavioral planner.
- An expert trajectory-guided Actor–Critic multi-objective optimization framework was improved for the motion control of quadruped robots. It incorporates a composite reward function to achieve hierarchical control under multi-task objectives, while integrating motion primitive imitation learning to accelerate policy convergence during training.

2. Methodology

Under the robot dynamics constraints and joint-level physical limits, the proposed framework aims to learn a mapping from the robot state space to continuous joint-level control commands, conditioned on the current system state (e.g., joint positions and velocities) and the given motion reference information. The learned policy outputs continuous and physically executable joint-level control signals, which are directly applied to the robot to stably track and compose multiple target motions under physical constraints. As illustrated in Figure 1, the imitation learning-based motion generation and control framework consists of two main components: (1) a motion generation module that produces reference motion sequences for tracking and (2) a motion control module that trains a policy network to achieve high-fidelity reproduction of the target motions.

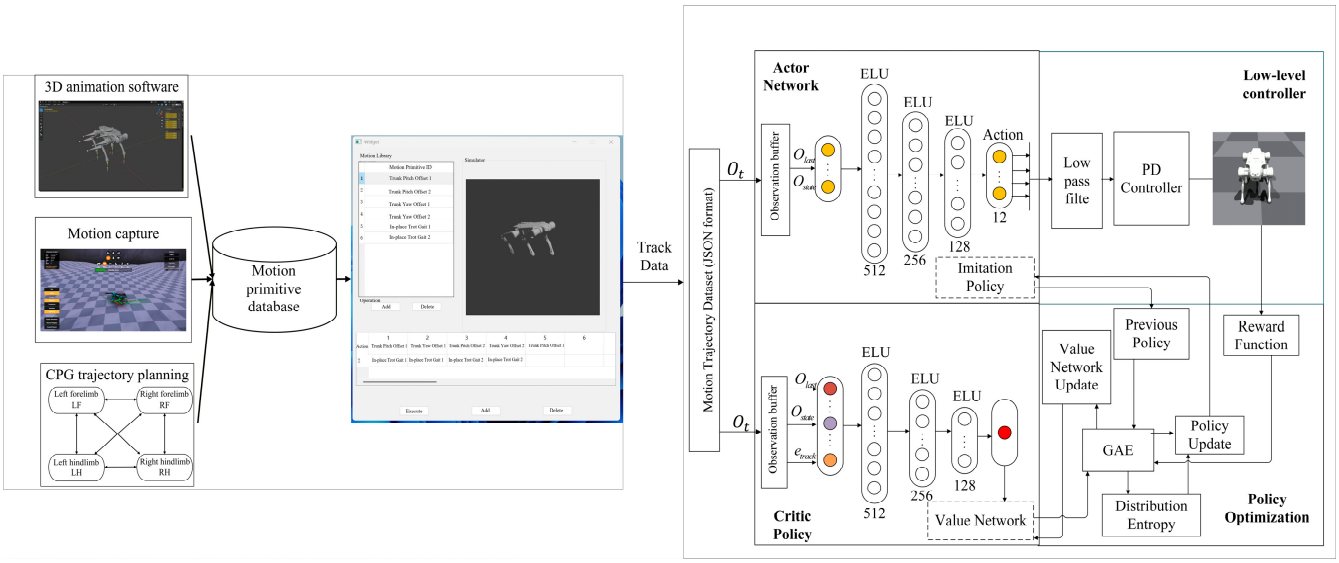


Figure 1. Motion generation and control framework.

2.1. Behavior Generation

2.1.1. Motion Design Based on a 3D Engine

Utilizing 3D simulation engines for quadruped robot motion design significantly reduces development costs and iteration cycles. Developers can detect and correct motion sequence flaws within the virtual environment. This process primarily consists of three components: modeling and skeletal rigging, keyframe motion design, and motion trajectory extraction and encapsulation, as previously shown in [31].

In the modeling and rigging phase, the kinematically consistent tree is first constructed through topological structuring, followed by geometric validation to ensure global coordinate system alignment and motion consistency. Finally, parametric conversion is performed using a customized plugin to reversely parse the URDF kinematic chain, achieving precise mapping between the skeleton and mechanical topology.

Keyframe motion design deconstructs actions into three phases: initiation, task execution, and termination. Multi-segment Bézier curves are employed to enable smooth transitions and trajectory generation. A custom plugin performs the extraction and encapsulation of motion trajectories. It precisely parses motion data and outputs it in structured CSV-formatted files containing key kinematic parameters such as torso, foot-end, and joint trajectories, along with timestamps.

2.1.2. Motion Capture Data Retargeting Based on Kinematic Chains

To enhance the biomimetic locomotion capabilities of quadruped robots by transferring high-fidelity motion capture data from quadruped animals, we adopt a cross-domain mapping framework based on sampled key points and inverse kinematics (IK), addressing the heterogeneity gap between the motion capture model and the robot. The specific steps are as follows:

- Size scaling: motion capture data was retargeted to the quadruped robot's kinematic chain through size scaling, eliminating geometric discrepancies between the source character and the target robot. This process calculates segment-specific length ratios to derive independent scaling factors for each kinematic sub-chain. For the quadruped robot utilized in this study, the selected scaling ratio was 0.725, with visual validation illustrated in Figure 2.
- Torso state determination: to resolve misalignment arising from mapping a flexible biological torso to a rigid robotic structure, four naturally symmetric limb connection

points were extracted from motion capture data—left/right shoulder joints (anterior) and left/right hip joints (posterior). A rigid-body transformation matrix was computed using the least squares method based on these reference points.

- Key points extraction: biomechanically representative key points were selected for kinematic chain mapping. Shoulder joints (forelimbs) and hip joints (hindlimbs) were extracted as limb root anchor points, corresponding spatially to the base mounting points of the robot's leg actuators. Their spatial coordinates were directly associated with the relative positional relationships in the torso rigid-body coordinate system, capturing 3D trajectory data of the limb endpoints.
- Inverse kinematics retargeting: biological key-point trajectories were converted into continuous joint-space motion commands. At each timestep t, the source motion specified the 3D position $\hat{\mathbf{x}}_i(t)$ of key point i. The corresponding target position $\mathbf{x}_i(\mathbf{q}_t)$ was determined by the robot's generalized coordinate pose $\mathbf{q}_t$. Inverse kinematics was then applied to construct a pose sequence $\mathbf{q}_{0:T}$ that tracks key points per frame, satisfying the following:

$$\text{argmin}_{\mathbf{q}_{0:T}} \sum_t \sum_i \|\hat{\mathbf{x}}_i(t) - \mathbf{x}_i(\mathbf{q}_t)\|^2 + (\overline{\mathbf{q}} - \mathbf{q}_t)^T \mathbf{W} (\overline{\mathbf{q}} - \mathbf{q}_t) \quad (1)$$

where **W** denotes a diagonal matrix specifying the regularization coefficients for each joint. One set of trajectories is shown in Figure 3.

- Post-processing: joint angle sequences output by inverse kinematics underwent validity screening and smoothing refinement. Simultaneously, centroid position adjustment was performed on motion capture results to ensure the center of mass coincided with the geometric center of the torso's rigid body.

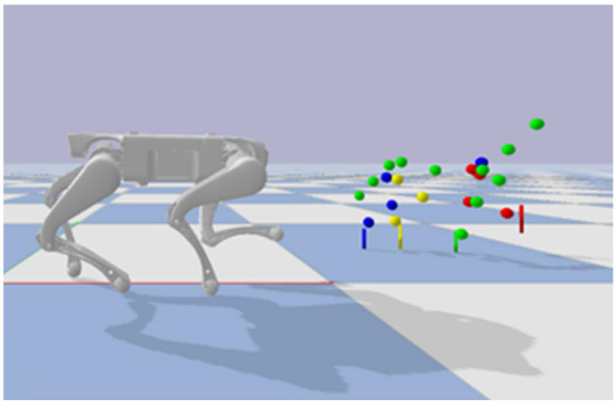


Figure 2. Effectiveness of the size scaling coefficients.

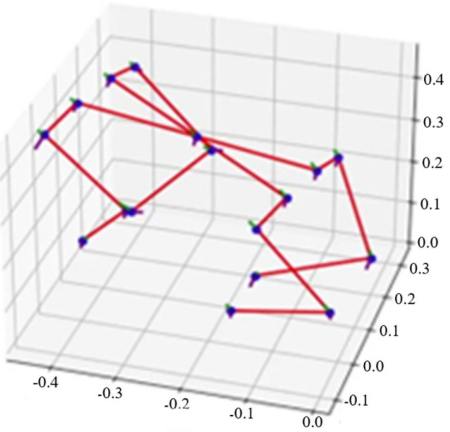


Figure 3. Visualization of the quadruped robot motion data.

2.1.3. Trajectory Planning-Based Rhythmic Locomotion Generation for Quadruped Robots

Central Pattern Generators (CPGs) are employed to construct a distributed neural oscillatory network, generating self-stabilizing rhythmic signals through the nonlinear phase dynamics of Hopf oscillator units.

The Hopf oscillator model is defined as follows:

$$\begin{cases} \dot{x} = \alpha(\mu - r^2)x - \omega y \\ \dot{y} = \alpha(\mu - r^2)y + \omega x \end{cases} \quad (2)$$

where x and y represent the state variables of the Hopf oscillator, whose dimensionality corresponds to the number of leg degrees of freedom. α denotes the convergence rate, μ specifies the oscillation amplitude, r is the intermediate parameter defined as $r^2 = x^2 + y^2$, and ω determines the steady-state oscillation frequency.

2.1.4. Design of the Behavior Planner

The behavior planner constructs a behavioral decision-making framework, enabling complex behavior generation through modular decomposition and dynamic composition mechanisms. Its core components mainly include:

- An action list management module based on hierarchical dynamic architecture

This module is responsible for converting motion primitives of different modalities and origins into digital objects that can be uniformly parsed and invoked by the system. It employs a hierarchical data structure to decouple and reconstruct key elements of motion primitives, including semantic descriptions, parameter configurations, and execution logic. This process forms structured storage units containing action name fields. This design facilitates subsequent behavior composition and execution, ensuring the flexibility of motion resources.

- A behavior script construction module based on temporal skill-chaining

This module transforms discrete action units into executable task sequences. The system constructs skill inheritance chains upon the completion of each action unit, enabling sequential playback and seamless transitions between actions. At the task sequence generation level, users combine action units via drag-and-drop operations within a visual editor, and the system automatically generates the corresponding behavior scripts.

- A behavior visualization module based on spatiotemporal synchronous mapping

Leveraging the 3D engine Blender, this module achieved synchronous co-evolution of the motion trajectories generated by the behavior planner along both temporal and spatial dimensions. Along the temporal dimension, actions were played sequentially according to their order on the timeline, clearly demonstrating the dynamic execution process of the behavior. Along the spatial dimension, the corresponding 3D motion postures were accurately reconstructed based on the specific parameters of each action unit.

The synergistic integration of the aforementioned modules formed a complete workflow, spanning from motion data import to behavior visualization. As illustrated in Figure 4, CSV files containing motion sequences were imported into the Action List Management Module via the “Add” option. Multiple motion sequences were then dragged into the Script Construction Module to form a more complex behavior sequence. Clicking “Play” subsequently generated the simulation animation within the Behavior Visualization Module.

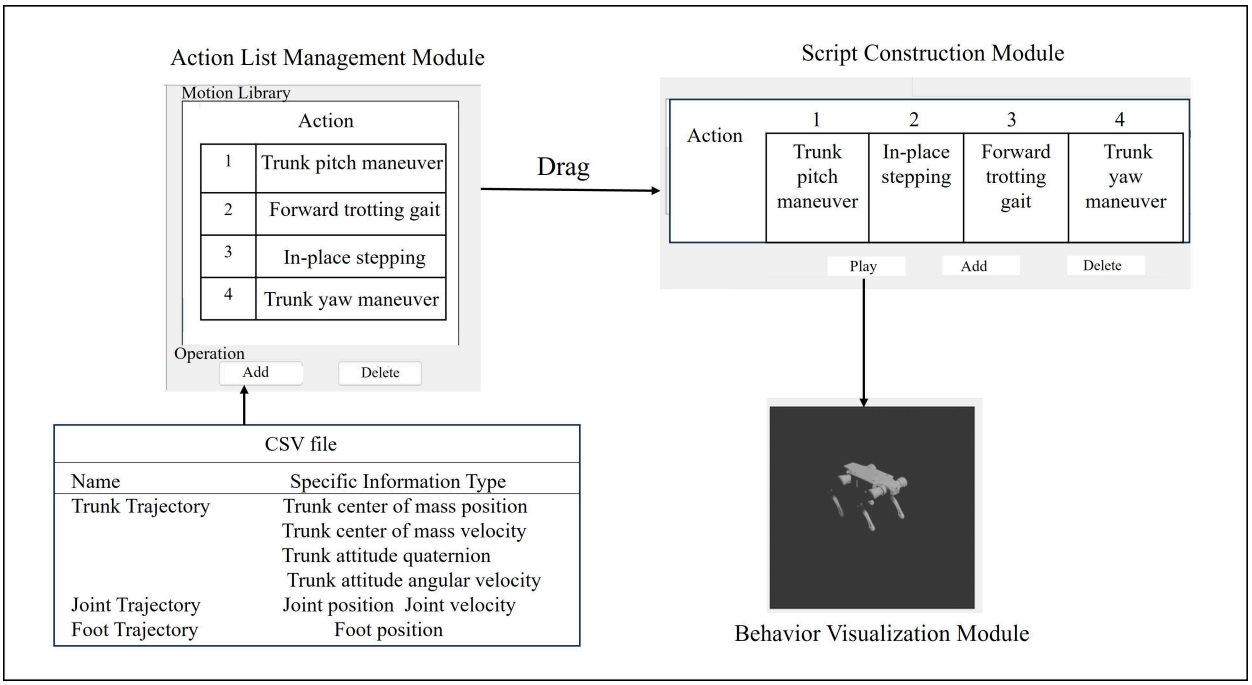


Figure 4. Overall workflow of the behavior planner.

2.2. Motion Control

This study formulates the motion imitation problem as a RL optimization task within the Markov Decision Process (MDP) framework, employing the Proximal Policy Optimization (PPO) algorithm. The core objective was to learn a control policy that enables the agent to maximize the expected return in a given task $J(\pi)$:

$$J(\pi) = E_{\tau \sim p(\tau|\pi)} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right] \tag{3}$$

where T denotes the time span of each episode, $\gamma \in [0, 1]$ represents the discount factor, and r_t indicates the instantaneous reward value. $p(\tau|\pi)$ is the probability of trajectory τ under policy π.

2.2.1. State and Action Space Design

To establish the imitation learning system architecture and to enable the efficient learning and generalization of complex motions by the policy network, the mathematical representation of the state space and action space must be addressed.

Within the quadruped robot imitation learning control system, the state space is structured based on the RL framework, integrating body IMU attitude features and joint motor feedback states. The normalized state vector is defined as $s_t = (q_{t-2:t}, a_{t-3:t-1})$, where $q_{t-2:t}$ represents the attitude angles over the previous three timesteps, primarily including the body roll, pitch, and yaw angles measured in real-time by the IMU and $a_{t-3:t-1}$ denotes the angular parameters of the 12 joint motors over the preceding three cycles. The action space adopts a joint position control strategy, with the target angles of the quadruped robot's 12 degree-of-freedom (DOF) joints serving as the core representation. The action space is defined as a 12-dimensional continuous vector $a = [\theta_1, \theta_2, \ldots, \theta_{12}]$, where each dimension corresponds to the target angle of a single joint.

2.2.2. Design of Reward Functions Based on Multi-Task Learning

This paper constructs a hierarchical and progressive reward function framework based on the statistical features of expert demonstration data and the kinematic–dynamic

characteristics of robots. It enforces the policy to replicate expert actions through trajectory tracking reward terms; introduces motion stability reward terms to mitigate the risk of instability under disturbances; and guides the policy to achieve safety assurances on the foundation of imitation via safety reward terms. The mathematical expression for the reward at each time step in the reward function was defined as follows:

$$r_t = r_t^{track} + r_t^{stab} + r_t^{safe} \tag{4}$$

where r_t^{track} denotes the trajectory tracking reward function, r_t^{stab} represents the motion stability reward function, and r_t^{safe} is the safety reward function.

The trajectory tracking reward function quantifies the similarity between policy-generated motion trajectories and expert demonstration data through mathematical measurement, thereby providing explicit gradient signals for policy optimization. Its specific mathematical expression is defined as follows:

$$r_t^{track} = \omega^p r_t^p + \omega^r r_t^r + \omega^{vl} r_t^{vl} + \omega^{va} r_t^{va} + \omega^e r_t^e + \omega^q r_t^q + \omega^{qv} r_t^{qv} \tag{5}$$

The constituent reward terms are defined as follows: centroid position reward r_t^p, centroid orientation reward r_t^r, centroid linear velocity reward r_t^{vl}, centroid angular velocity reward r_t^{va}, foot-end position reward r_t^e, joint position reward r_t^q, and joint velocity reward r_t^{qv}. Each reward function adopts the exponential quadratic form $r = \exp\left[-\alpha\|\hat{x} - x\|^2\right]$, where tunable hyperparameter α regulates the reward sensitivity. Here, x and $\hat{x}$ denote the actual state and desired expert state, respectively.

The motion stability reward function mathematically quantifies deviations between the robot's torso attitude, centroid motion state, and stable equilibrium objectives, thus guiding the policy network to generate disturbance-resistant motion patterns. Its comprehensive expression is defined as follows:

$$r_t^{stab} = \lambda_{\text{ori}} \cdot \cos(\hat{z}_B, \hat{z}_W) - \lambda_{\text{com}} \cdot d_{\text{com}} \tag{6}$$

where $\hat{z}_B, \hat{z}_W$ represents the torso attitude angles and the gravity-aligned z-axis unit vector, d_{com} indicates the projection distance of the centroid within the support polygon, and $\lambda_{\text{ori}}, \lambda_{\text{com}}$ are the corresponding weighting coefficients.

The safety reward function enforces mathematical constraints to ensure critical parameters, including the joint positions, velocities, and torques of output actions, which consistently remained within physical viability and system safety thresholds.

$$r_t^{safe} = -\left(\sum_{i=1}^{12}\left[k_p \cdot (\theta_i - \theta_{\text{threshold}})^2 + \exp\left(\frac{\left|\dot{\theta}_i\right|}{\dot{\theta}_{\max}}\right) - 1 + \lambda_\tau \cdot \left(\frac{|\tau_i|}{\tau_{\max}}\right)^3\right]\right) \tag{7}$$

where $\theta_{threshold} = \theta_{\min} + \delta$ denotes soft constraint thresholds, $\delta = 0.1rad$ represents a buffer zone to prevent policy oscillation induced by abrupt clamping, $\dot{\theta}_{\max}$ and $\tau_{\max}$ specify the maximum allowable joint velocity and torque, respectively, and k_p, λ_τ are corresponding proportional coefficients.

2.2.3. Imitation Learning Network

Integrating all components of the imitation learning control system yielded the overall architecture of the quadruped robot imitation learning controller. This framework is primarily divided into four components: Actor network, Critic network, low-level controller, and policy optimization.

The Actor network module enables the controller to regulate the physical model of robots. As depicted in Figure 5, the Actor network accepts 42-dimensional input vectors consisting primarily of raw environmental observations, specifically selected as $\left[v_{ang}, G, q_j, \dot{q}_j, a_{last}\right] \in R^{42}$. The information is typically input into the policy network in the form of low-dimensional vectors, which accelerate the training process and improve the convergence efficiency of the policy. The complete set of observations is shown in Table 1. Within the Isaac Gym training environment, body pose sensors, historical joint sensors, and joint position sensors were implemented to collect raw observational data. These observations were subsequently stored in a history buffer, where data from each sensor were packaged across three consecutive timesteps (t, t − 1, t − 2) and sequentially concatenated to serve as inputs for the imitation learning policy.

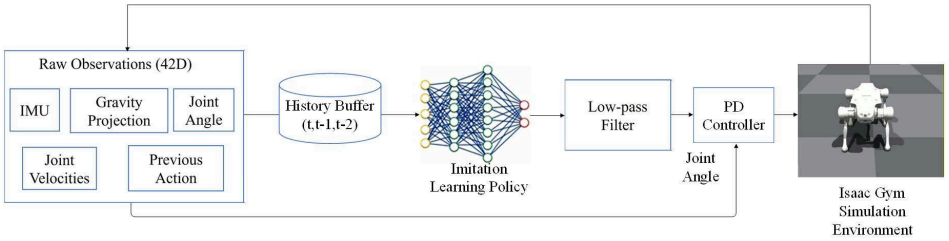


Figure 5. Imitation Learning Policy Controller.

Table 1. Observations of the Policy Network.

Symbol	Dimension	Description
v_{ang}	3	Angular velocity of the center of mass (COM)
G	3	Projection of gravitational force
q_j	12	Joint angle
$\dot{q}_j$	12	Joint velocity
a_{last}	12	Action at the previous time step

The Critic network module must balance efficient representation of dynamic environments with stability requirements for policy optimization, as illustrated in the control framework (Figure 6). For quadruped robot locomotion control, the Critic network's input feature space comprises $\left[v_{lin}, v_{ang}, G, q_j, \dot{q}_j, a_{last}, e_{errot}\right] \in R^{93}$. The details of the tracking error are presented in Table 2, typically including proprioceptive information and task-specific high-level goal parameters. Unlike the Actor network that relies solely on current state observations, the Critic network explicitly models the dynamic characteristics of goal deviation during policy execution by incorporating tracking error information, thereby introducing task-oriented long-term reward mechanisms into value estimation.

Table 2. Tracking Errors in the Privileged Observations.

Symbol	Dimension	Description
CoM_pos_error	3	Tracking error of COM'S position, relative to the world coordinate system
CoM_euler_error	3	Tracking error of COM's Euler angle
CoM_lin_error	3	Tracking error of COM's linear velocity
CoM_ang_error	3	Tracking error of COM's angular velocity
toe_pos_error	12	Tracking error of COM'S position, relative to the world coordinate system
dof_pos_error	12	Joint position tracking error
dof_vel_error	12	Joint velocity tracking error

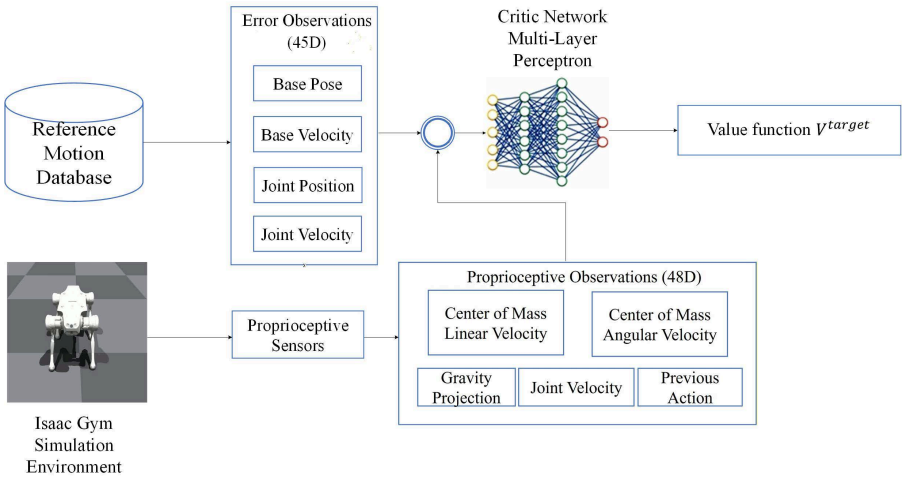


Figure 6. Control Framework of Critic Network.

The low-level controller converts joint position increments from the Actor network into desired rotation commands. These commands are processed by a Proportional-Derivative (PD) controller to compute joint torques for motor actuation, with the control torque formulated as follows:

$$u(t) = K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt} \tag{8}$$

where K_p and K_d represent the proportional and derivative gains, respectively, $e(t)$ denotes position error, and $de(t)/dt$ indicates velocity error. To simplify control objectives and enhance stability, the desired joint velocity was typically set to zero.

PPO is an optimization algorithm based on the Actor–Critic architecture, whose core lies in the update mechanisms for the policy and value networks. Upon receiving control commands, the agent utilizes current state information to compute the instantaneous reward signal R through the reward function. The Critic network outputs an estimate of the state-value function V^{target}. This estimate is combined with R to compute the advantage estimate and quantify the discrepancy between the value function prediction and the actual reward, thereby providing comprehensive feedback for policy evaluation.

The advantage estimation adopts the form of generalized advantage estimation (GAE), denoted as $\hat{A}_t^{\mathrm{GAE}}$. This method employs an adjustable parameter λ to perform an exponentially weighted fusion of Temporal Difference (TD) errors across different step lengths, establishing a continuously tunable balance mechanism between estimation bias and variance. The K-step TD error δ_t^V is defined as follows:

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{9}$$

where $v(s)$ represents the state-value function, γ denotes the discount factor, and t is the timestep index. GAE computes an exponentially weighted average of K-step TD errors, so a compact form of GAE $\hat{A}_t^{\mathrm{GAE}}$ can be derived through series expansion, as follows:

$$\hat{A}_t^{\mathrm{GAE}} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}^V \tag{10}$$

The statistical properties of GAE are jointly determined by the discount factor γ and λ. γ controls the discounting of future returns, influencing the focus on long-term gains. λ regulates the bias–variance trade-off intensity. In most continuous control tasks, setting λ = 0.95 yields empirically optimal performance.

The Critic network takes proprioceptive observations and error observations as inputs. After processing through a three-layer neural network, it outputs the estimated state function V^{target} for the given state. The objective is to approximate V^{target}, providing a benchmark for policy evaluation and subsequently guiding the policy updates of the Actor network. The update of the Critic network relies on the accurate estimation of V^{target}, aiming to minimize the mean squared error between the predicted values and the target values. The target value is computed using the TD (λ) method, which extends Temporal Difference (TD) learning by introducing eligibility traces to assign credit for rewards. The TD (λ) update rule is given by the following:

$$V(s_t) \leftarrow V(s_t) + \alpha \delta_t E_t \tag{11}$$

where $V(s_t)$ is the estimated value of the current state, α is the learning rate, δ_t is the TD error at timestep t, and E_t is the eligibility trace at timestep t. This process drives the value function toward the true value by minimizing the mean squared error between the predicted and target values.

2.2.4. Domain Randomization and Disturbances

To improve robustness against sim-to-real discrepancies and cross-source motion variations, we incorporated domain randomization and observation noise injection during training.

Specifically, key physical parameters, including mass, inertia, actuator-related parameters, and friction coefficients, were randomly perturbed within predefined ranges, as summarized in Table 3. In addition, random offsets were applied to the initial states, including joint configurations and Center of Mass (CoM) states, to account for initialization uncertainties.

Table 3. Domain randomization of physical parameters.

Item	Range
Mass	[0.8, 1.2] × default
Inertia	[0.5, 1.5] × default
Motor torque scaling	[0.8, 1.2] × default
PD gain scaling	[0.8, 1.2] × default
Initial joint position scaling	[0.5, 1.5] × default
Initial CoM position scaling	[0.8, 1.2] × default
Ground friction	[0.1, 2.5]
Motor friction	[0, 0.05]

Furthermore, observation noise was injected to model measurement noise and observation uncertainty. The noise magnitudes applied to joint states, CoM velocities, and gravity projection are listed in Table 4. These randomization and noise injection strategies collectively improved the stability and generalization of the learned whole-body control policy under uncertainties.

Table 4. Observation noise settings.

Item	Noise Magnitude
Joint position noise	0.01
Joint velocity noise	1.5
CoM linear velocity noise	0.1
CoM angular velocity noise	0.2
Gravity projection noise	0.05

3. Experimental Tests and Results

This study employed the Lite3 quadruped robot model developed by DeepRobotics (Hangzhou, China), conducting training on the Isaac Gym simulation platform with an

NVIDIA GeForce RTX 4090 GPU. Subsequently, the trained policy underwent dynamic adaptability validation for the robot model in the PyBullet physics engine. Additionally, the Blender 4.1 3D animation engine was utilized to create quadruped robot motion animations via keyframe insertion, facilitating motion sequence design and export. For RL, the PPO algorithm was adopted due to its computational efficiency. The hyperparameter configurations used during PPO-based simulation training are summarized in Table 5.

Table 5. Hyperparameter configurations for PPO during simulated training.

Parameter	Value
Learning rate	$2 \times 10^{-5}/1 \times 10^{-5}$
Discount factor γ	0.99
Batch size	$98,304(4096 \times 24)$
Mini-batch size	$24,576(4096 \times 6)$
Policy clip range	0.2
Entropy coefficient	0.01
GAE λ	0.95
Target KL divergence	0.01

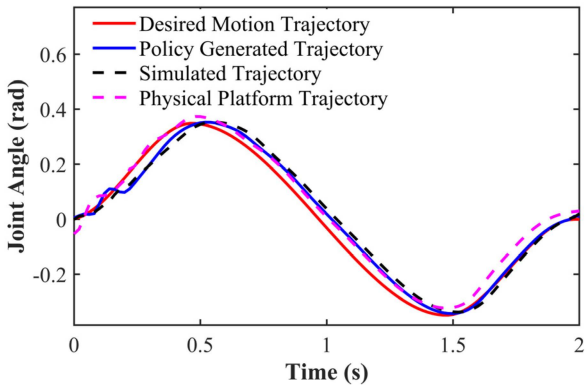
In the experiments, the centroid height of the quadruped robot in standing posture was set to 0.32 m. The joint angle vector was initialized to $q = [0, \ -0.773, \ 1.5]^{\mathrm{T}}$ for hip, thigh, and calf joints, respectively. To ensure that joint rotation remained within safe operating limits, the upper and lower bounds of the joint motion range are detailed in Table 6.

Table 6. Joint motion range contraints.

Joint Type	Dimension	Thigh Joint (rad)	Calf Joint (rad)
Upper limit	0.523	0.314	2.792
Lower limit	-0.523	-2.67	0.524

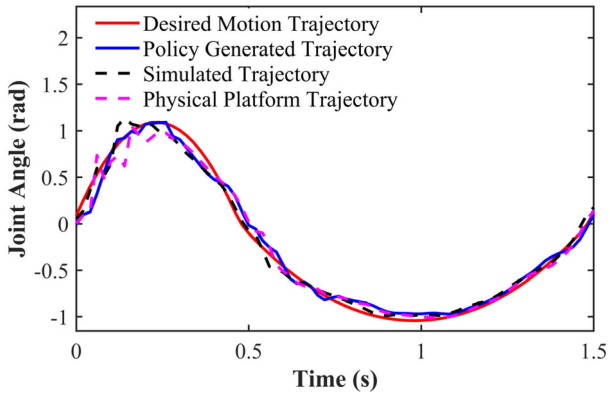
3.1. Motion Design Validation Using a 3D Engine

The experiment utilized Blender to create motion sequences, incorporating roll, pitch, and yaw rotations, with target angles set at $\pm 20^{\circ}$ in both directions. Motion control policies were generated in parametric space through imitation learning, with trajectory tracking accuracy verified via PyBullet simulation and the Lite3 physical platform. Figure 7 presents comparative results of Euler angle tracking performance. Quantitative analysis shows the maximum tracking errors of 0.03 rad (roll), 0.01 rad (pitch), and 0.01 rad (yaw), meeting precision requirements for motion control.

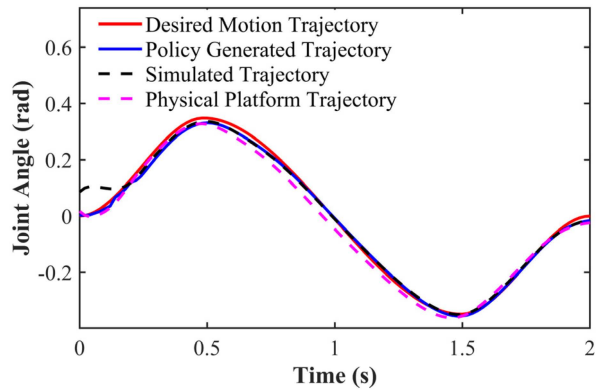


(**a**) Roll motion

Figure 7. *Cont.*



(**b**) Pitch motion



(**c**) Yaw motion

Figure 7. Euler angle tracking performance of the robot's Center of Mass (COM). The curves compare the desired trajectories with the policy-generated, simulated, and physical platform responses for (**a**) Roll, (**b**) Pitch, and (**c**) Yaw motions.

3.2. Experimental Validation of Motion Capture-Based Motion Repositioning

In quadruped motion repositioning research, we first acquired the reference trajectory data adapted for the Lite3 robot by performing inverse kinematic mapping on motion-captured trot gait patterns, validating the effectiveness of forward trot motion (whose action sequence is shown in Figure 8). Subsequently, through inverse kinematics parameter inversion, a mirrored backward trot motion sequence and control strategy were generated (with motion snapshots presented in Figure 9).

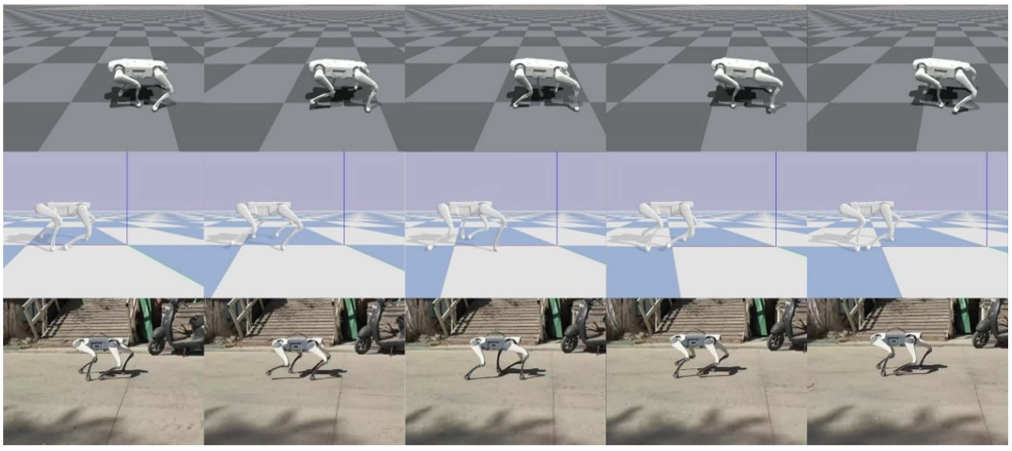


Figure 8. Repositioned trot gait snapshots. From top to bottom, the snapshots correspond to Isaac Gym, PyBullet, and real-world environments, respectively.

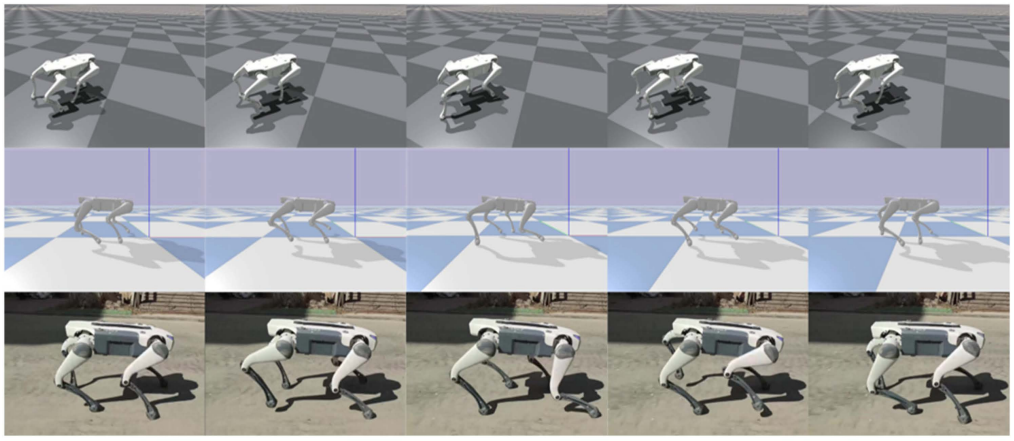
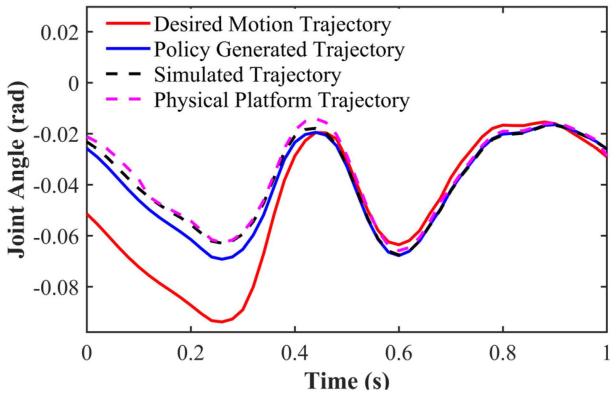
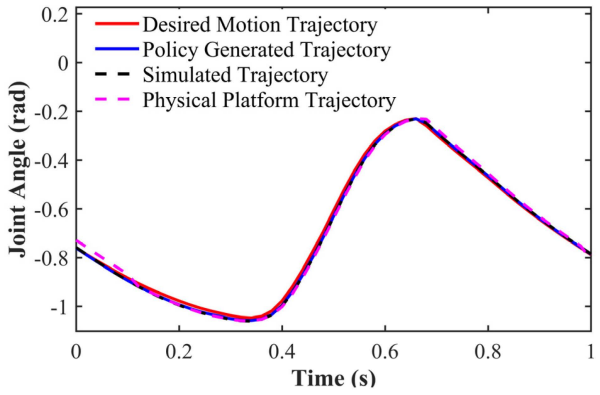


Figure 9. Snapshots of repositioned backward trot motion. From top to bottom, the snapshots correspond to Isaac Gym, PyBullet, and real-world environments, respectively.

We recorded the three DOF joint rotation angles in the robot's right forelimb. As illustrated in Figure 10, the complete gait cycle analysis demonstrates that trained, simulated, and experimentally measured trajectories all effectively track target joint angles, while exhibiting excellent continuity and smoothness without observable step distortion. Notably, the hip joint's initial rotation displays an inward flexion tendency due to training-phase configurations designed to ensure motion initiation continuity. Figure 11 further presents the joint angle tracking performance of the hip, thigh, and calf joints during 5 s periodic motions, with quantitative data confirming the effective reference angle across all three joints.

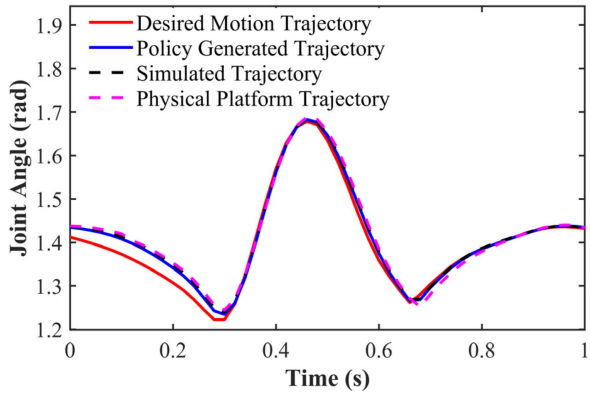


(**a**) Hip joint



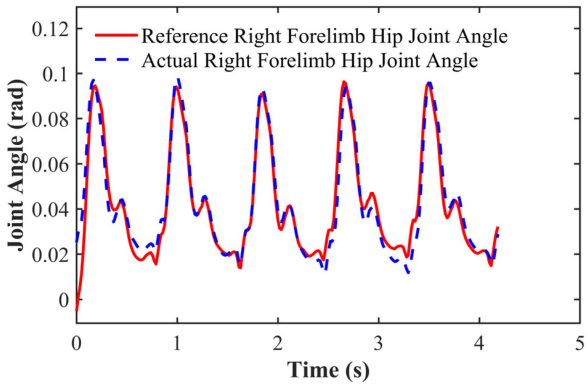
(**b**) Thigh joint

Figure 10. *Cont.*

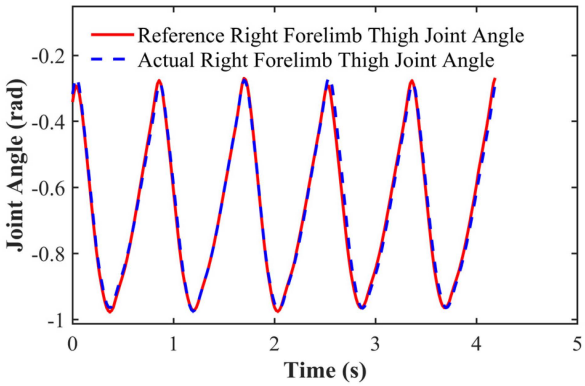


(**c**) Calf joint

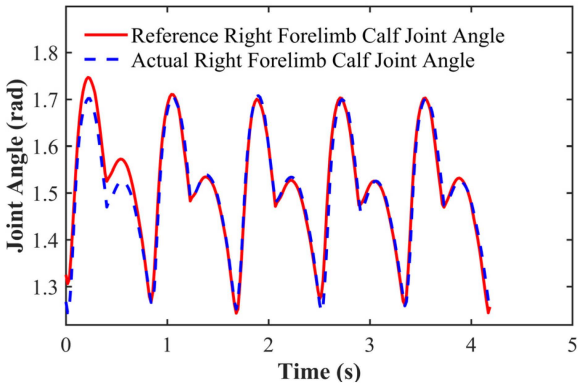
Figure 10. Joint angle tracking curves of the right foreleg.



(**a**) Hip joint



(**b**) Thigh joint



(**c**) Calf joint

Figure 11. Joint angle variation curves of the right foreleg.

3.3. Sim-to-Real Deployment and Experimental Verification

To evaluate sim-to-real transfer performance, the policy trained in simulation was directly deployed on the Lite3 quadruped robot without any additional fine-tuning. During training, limited domain randomization and observation noise were introduced to improve robustness to real-world uncertainties, as described in Section 2.2.4.

After training converges, the PyTorch 1.13.1 actor network was serialized into a static TorchScript model (.pt) and transferred from the training workstation to the robot's onboard computer (NVIDIA Jetson Orin NX) via a secure network interface (SSH). Onboard, the policy was loaded using C++/LibTorch for online inference, with observation normalization and state processing kept consistent with simulation.

The real-world control system follows a low-rate policy inference–high-rate execution architecture. The policy runs at 50 Hz and outputs joint position residuals, which are mapped to target joint positions and tracked by a 1 kHz low-level PD controller. A state-based safety failsafe was implemented to ensure hardware safety during experiments.

To quantitatively assess sim-to-real transfer fidelity, simulated (Sim) and real-world (Real) trajectories were compared under identical command inputs. The discrepancy between simulation and real execution was evaluated using three standard metrics: root mean square error (RMSE), mean absolute error (MAE), and maximum absolute error (Max Error).

Specifically, errors of the robot CoM attitude, represented by Roll, Pitch, and Yaw, are summarized in Table 7. In addition, joint-level errors for Hip, Thigh, and Knee are reported in Table 8 to characterize sim-to-real discrepancies at the actuator execution level.

Table 7. Quantitative Sim-to-Real Errors of Robot CoM Euler Angles.

Attitude	RMSE (rad)	MAE (rad)	Max Error (rad)
Roll	0.0398	0.0343	0.0763
Pitch	0.0974	0.0575	0.5145
Yaw	0.0315	0.0226	0.1075

Table 8. Quantitative Sim-to-Real Errors of Robot Joint States.

Joint	RMSE (rad)	MAE (rad)	Max Error (rad)
Hip	0.0057	0.0043	0.0146
Thigh	0.0386	0.0331	0.0830
Knee	0.0324	0.0230	0.0872

The quantitative errors reported in Tables 7 and 8 are computed from the corresponding tracking trajectories presented in the subsequent experimental results. In particular, the CoM attitude errors are derived from the yaw motion shown in Figure 7, whereas the joint-state errors are calculated based on the right foreleg joint trajectories shown in Figures 10 and 11.

As shown in Tables 7 and 8, the discrepancies between simulated and real robot executions remain within a small and bounded range across all reported metrics. Specifically, the RMSE and MAE of the robot CoM Euler angles were below 0.1 rad, whereas the corresponding maximum absolute errors remain limited. Similarly, for key joint states, both RMSE and MAE remain below 0.04 rad, with bounded maximum errors. Overall, these results indicate that the simulation results closely reflect the execution behavior on real hardware.

3.4. Experimental Validation of CPG-Based Trajectory Planning Motion Design

Taking the CPG-generated in-place stepping motion as an example, a phase-coupled oscillator model was employed to establish parametric equations for foot-end trajectories, with a gait cycle of 0.5 s and a leg lift height of 0.1 m. Figure 12 demonstrates the physical control performance, where the robot accurately replicates reference trajectories, validating the framework's effectiveness for CPG-planned motions. This controller maintains consistency with the previously described Blender and motion capture-based controllers.

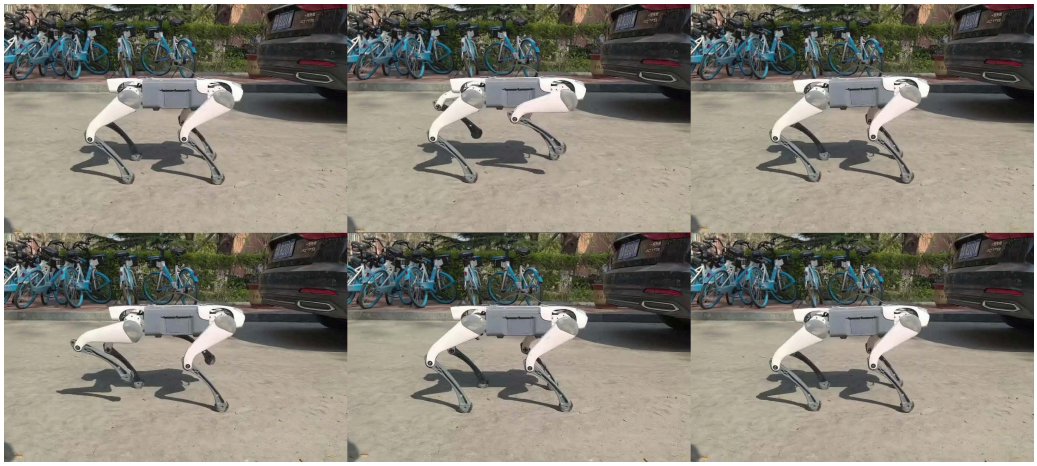


Figure 12. Quadruped robot in-place stepping experiment.

3.5. Multi-Action Composite Behavior Experiment

Building upon the validated single-task control architecture, we developed an imitation learning-based multi-task control system. This system integrates Blender-generated torso twisting, motion-captured trot repositioning (forward/backward), and CPG-based in-place stepping into a unified behavioral dataset. Figure 13 presents the physical robot executing this multi-action sequence, visually confirming the imitation learning framework's capability for composite behavior generation.



Figure 13. Multi-task control implementation snapshots.

3.6. Ablation Study on Temporal Behavior Planning

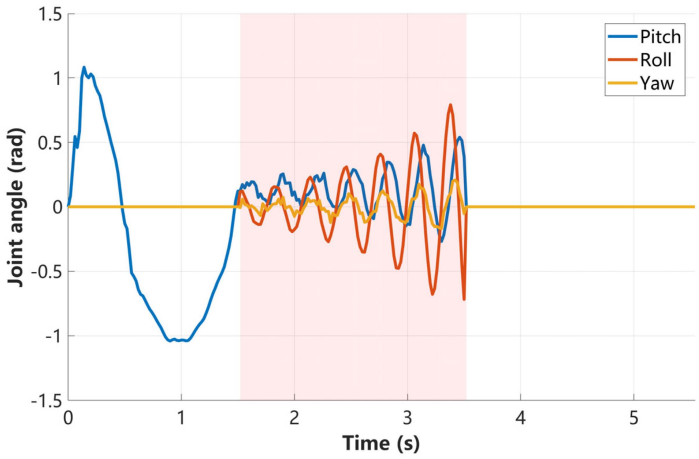
To isolate the contribution of the proposed temporal behavior planner, we conducted an ablation study, in which all compared methods shared the same imitation-learning-based low-level controller. All methods were evaluated under identical dataset splits, experimental settings, evaluation metrics, and on the same physical hardware platform, ensuring a fair and controlled comparison. We compared our behavior-planner-based motion primitive transition approach with two motion switching strategies commonly used in practice:

Method A: direct switching, where motion primitives are concatenated without any temporal smoothing or state alignment;

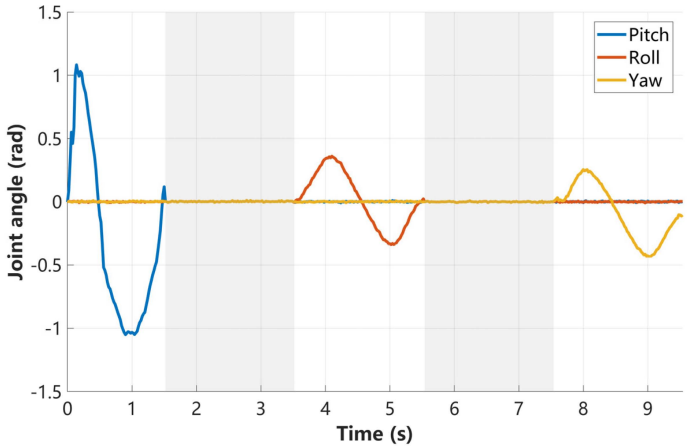
Method B: reset-to-neutral execution, where the robot returns to a stable standing pose and waits for stabilization before executing the next primitive [18,28];

Method C: continuous behavior planning (ours), which explicitly synthesizes smooth transition trajectories in the temporal domain.

Figure 14 shows the variation of the robot's CoM Euler angles under different motion switching strategies. As shown in Figure 14a, Method A lacks effective continuity when transitioning from the previous motion to the next, resulting in poorly controlled attitude changes and preventing the robot from successfully completing subsequent actions. As shown in Figure 14b, Method B restores the robot to an initial standing posture before executing the next motion, enabling the completion of the entire motion sequence.

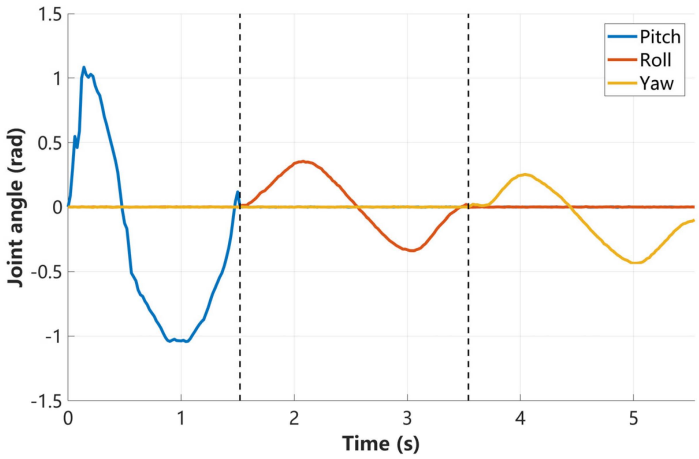


(**a**) Direct switching



(**b**) Reset-to-neutral execution

Figure 14. *Cont.*



(**c**) Continuous behavior planning

Figure 14. Attitude response comparisons under different motion primitive transition strategies.

In contrast, as shown in Figure 14c, Method C (ours) maintains continuity between consecutive motions during switching, keeping the attitude changes smooth. The roll, pitch, and yaw angles vary smoothly over time, allowing the motion sequence to be completed continuously.

Overall, although both the proposed method and the reset-based baseline were able to complete the task without instability, our approach achieves substantially smoother execution and higher efficiency by eliminating unnecessary waiting phases. In contrast, direct switching consistently fails due to the lack of effective continuity between consecutive motions.

Since all methods employ the same low-level controller, the observed performance differences can be solely attributed to the motion primitive transition strategy. These results suggest that the temporal behavior planner is beneficial for achieving stable, continuous, and efficient multi-skill execution on real robotic systems.

4. Conclusions

This paper proposes a quadruped robot motion strategy generation method based on multimodal motion primitives and imitation learning. The method constructs a three-level architecture of "motion primitives–skill chains–complex behaviors", enabling parameterized design under joint constraints, biological feature transfer, and tunable trajectory generation. Meanwhile, a multi-task control framework based on imitation learning was designed, which adopts the Actor–Critic architecture and a hierarchical composite reward function. A training–simulation–deployment verification system was built based on Blender and physical entity platforms, systematically verifying the generation of basic gaits and the effect of motion transfer, as well as the execution efficiency and generalization ability of dynamic behavior sequences under multi-task control.

It should be emphasized that the proposed method was evaluated under structured task settings with predefined motion primitives and controlled execution conditions. Although the results confirm its capability in generating and composing multimodal motions, its performance in highly dynamic tasks and unstructured environments has not yet been systematically investigated. Future work will focus on incorporating dynamics-aware optimization and robustness enhancement to extend the applicability of the framework to more complex real-world scenarios.

Author Contributions: Conceptualization and methodology, Q.Z.; software, G.L. and B.L.; writing—original draft preparation, G.L.; writing—review and editing, C.L.; data curation, C.Z.; funding acquisition, Q.Z. and H.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Key Research and Development Program of China, grant number 2024YFB4708700, the Key R&D Program of Shandong Province, China, grant number 2025CXGC010214, and the Shandong Provincial Natural Science Foundation, grant number ZR2022MF296.

Data Availability Statement: The original contributions presented in this study are included in the article material. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhang, W.; Xu, S.; Cai, P.; Zhu, L. Agile and Safe Trajectory Planning for Quadruped Navigation with Motion Anisotropy Awareness. In Proceedings of the 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Abu Dhabi National Exhibition Center, United Arab Emirates, 14–18 October 2024; pp. 8839–8846.
2. Cruz Ulloa, C.; Cerro, J.; Barrientos, A. Mixed-reality for Quadruped-robotic Guidance in SAR Tasks. *J. Comput. Des. Eng.* **2023**, *10*, 1479–1489. [CrossRef]
3. Li, J.; Liu, Z.; Li, S.; Jiang, J.; Li, Y.; Tian, C.; Wang, G. Motion Planning for a Quadruped Robot in Heat Transfer Tube Inspection. *Autom. Constr.* **2024**, *168*, 105753. [CrossRef]
4. Spot–the Agile Mobile Robot. Available online: https://bostondynamics.com/products/spot/ (accessed on 11 September 2023).
5. Xie, Z.; Clary, P.; Dao, J.; Morais, P.; Hurst, J.; Panne, M. Learning Locomotion Skills for Cassie: Iterative Design and Sim-to-Real. In Proceedings of the Conference on Robot Learning, Osaka, Japan, 30 October–1 November 2019; p. 100.
6. Medeiros, V.S.; Jelavic, E.; Bjelonic, M.; Siegwart, R.; Meggiolaro, M.A.; Hutter, M. Trajectory Optimization for Wheeled-legged Quadruped Robots Driving in Challenging Terrain. *IEEE Robot. Autom. Lett.* **2020**, *5*, 4172–4179. [CrossRef]
7. Liu, H.; Yuan, Q. Safe and Robust Motion Planning for Autonomous Navigation of Quadruped Robots in Cluttered Environments. *IEEE Access* **2024**, *12*, 69728–69737. [CrossRef]
8. Song, Z.; Yue, L.; Sun, G.; Ling, Y.; Wei, H.; Gui, L.; Liu, Y.-H. An Optimal Motion Planning Framework for Quadruped Jumping. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*; IEEE: New York, NY, USA, 2022; pp. 11366–11373.
9. Ju, Z.; Wei, K.; Jin, L.; Xu, Y. Investigating Stability Outcomes Across Diverse Gait Patterns in Quadruped Robots: A Comparative Analysis. *IEEE Robot. Autom. Lett.* **2024**, *9*, 795–802. [CrossRef]
10. Yao, Q.; Wang, J.; Yang, S.; Wang, C.; Zhang, H.; Zhang, Q.; Wang, D. Imitation and Adaptation Based on Consistency: A Quadruped Robot Imitates Animals from Videos Using Deep Reinforcement Learning. In *2022 IEEE International Conference on Robotics and Biomimetics (ROBIO)*; IEEE: New York, NY, USA, 2022; pp. 1414–1419.
11. Fawcett, R.T.; Afsari, K.; Ames, A.D.; Hamed, K.A. Toward a Data-Driven Template Model for Quadruped Locomotion. *IEEE Robot. Autom. Lett.* **2022**, *7*, 7636–7643. [CrossRef]
12. Li, T.; Zhang, Y.; Zhang, C.; Zhu, Q.; Sheng, J.; Chi, W.; Zhou, C.; Han, L. Learning Terrain-adaptive Locomotion with Agile Behaviors by Imitating Animals. In Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023; IEEE: New York, NY, USA; pp. 339–345.
13. Kim, D.; Carlo, J.D.; Katz, B.; Bledt, G.; Kim, S. Highly Dynamic Quadruped Locomotion via Whole-body Impulse Control and Model Predictive Control. *arXiv* **2019**, arXiv:1909.06586. [CrossRef]
14. Gurram, M.; Uttam, P.K.; Ohol, S.S. Reinforcement Learning for Quadruped Locomotion: Current Advancements and Future Perspectives. In Proceedings of the 2025 9th International Conference on Mechanical Engineering and Robotics Research (ICMERR), Barcelona, Spain, 15–17 January 2025; pp. 28–38.
15. Shi, H.; Zhou, B.; Zeng, H.; Wang, F.; Dong, Y.; Li, J.; Wang, K.; Tian, H.; Meng, M.Q.-H. Reinforcement Learning with Evolutionary Trajectory Generator: A General Approach for Quadruped Locomotion. *IEEE Robot. Autom. Lett.* **2022**, *7*, 3085–3092. [CrossRef]
16. Bledt, G.; Wensing, P.M.; Kim, S. Policy-regularized Model Predictive Control to Stabilize Diverse Quadruped Gaits for the MIT Cheetah. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, Vancouver, BC, Canada, 24–28 September 2017; pp. 4102–4109.
17. Lee, J.; Hwangbo, J.; Wellhausen, L.; Koltun, V.; Hutter, M. Learning Quadruped Locomotion over Challenging Terrain. *Sci. Robot.* **2020**, *5*, eabc5986. [CrossRef] [PubMed]
18. Hwangbo, J.; Lee, J.; Dosovitskiy, A.; Bellicoso, D.; Tsounis, V.; Koltun, V.; Hutter, M. Learning Agile and Dynamic Motor Skills for Legged Robots. *Sci. Robot.* **2019**, *4*, eaau5872. [CrossRef] [PubMed]
19. Xie, Z.; Da, X.; Babich, B.; Garg, A.; van de Panne, M. Glide: Generalizable Quadruped Locomotion in Diverse Environments with a Centroidal Model. In *International Workshop on the Algorithmic Foundations of Robotics*; Springer International Publishing: Cham, Switzerland, 2022; pp. 523–539.
20. Azimi, D.; Hoseinnezhad, R. Hierarchical Reinforcement Learning for Quadruped Robots: Efficient Object Manipulation in Constrained Environments. *Sensors* **2025**, *25*, 1565. [CrossRef] [PubMed]

21. Peng, X.B.; Abbeel, P.; Levine, S.; Van de Panne, M. DeepMimic: Example-guided Deep Reinforcement Learning of Physics-based Character Skills. *ACM Trans. Graph.* **2018**, *37*, 143.1–143.14. [CrossRef]
22. Yang, R.; Chen, Z.; Ma, J.; Zheng, C.; Chen, Y.; Nguyen, Q.; Wang, X. Generalized Animal Imitator: Agile Locomotion with Versatile Motion Prior. In Proceedings of the Computing Research Repository, Honolulu, HI, USA, 11–16 May 2024; pp. 4631–4650.
23. Roh, S.G. Rapid Speed Change for Quadruped Robots via Deep Reinforcement Learning. In Proceedings of the 2023 IEEE International Conference on Development and Learning (ICDL), Macau, China, 9–11 November 2023; pp. 473–478.
24. Chen, S.; Zhang, B.; Mueller, M.W.; Rai, A.; Sreenath, K. Learning Torque Control for Quadruped Locomotion. In Proceedings of the 2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids), Austin, TX, USA, 12–14 December 2023; pp. 1–8.
25. Wang, Z.; Cheng, X.; Zhuo, Z.; Jia, W.; Huang, K.; Jiang, J. Biomimetic Intelligent Motion Control Method for Quadruped Robot with Manipulator. In Proceedings of the 2024 10th International Conference on Mechanical and Electronics Engineering (ICMEE), Xi'an, China, 27–29 December 2024; pp. 238–244.
26. Miki, T.; Lee, J.; Hwangbo, J.; Wellhausen, L.; Koltun, V.; Hutter, M. Learning Robust Perceptive Locomotion for Quadruped Robots in the Wild. *Sci. Robot.* **2022**, *7*, eabk2822. [CrossRef] [PubMed]
27. Ding, P.; Zhao, H.; Zhang, W.; Song, W.; Zhang, M.; Huang, S.; Yang, N.; Wang, D. Quar-Vla: Vision-language-action Model for Quadruped Robots. In *European Conference on Computer Vision*; Springer Nature Switzerland: Cham, Switzerland, 2024; pp. 352–367.
28. Bordalba, R.; Schoels, T.; Ros, L.; Porta, J.M.; Diehl, M. Direct collocation methods for trajectory optimization in constrained robotic systems. *IEEE Trans. Robot.* **2022**, *39*, 183–202. [CrossRef]
29. Ibarz, J.; Tan, J.; Finn, C.; Kalakrishnan, M.; Pastor, P.; Levine, S. How to train your robot with deep reinforcement learning: Lessons we have learned. *Int. J. Robot. Res.* **2021**, *40*, 698–721. [CrossRef]
30. Correia, A.; Alexandre, L.A. A survey of demonstration learning. *Robot. Auton. Syst.* **2024**, *182*, 104812.
31. Luo, C.; Zhang, Q.; Li, S.; Chai, H.; Xu, W.; Wang, K. Behavior Generation Approach for Quadruped Robots Based on 3D Action Design and Proximal Policy Optimization. In Proceedings of the 2024 IEEE International Conference on Unmanned Systems (ICUS), Nanjing, China, 18–20 October 2024; pp. 1088–1093.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.